

NVIDIA Deep Learning Software Engineer Interview Review

Algorithmic Model Optimization (LLMs, Diffusion, TensorRT, TRT-LLM)

Role-Focused Quick Outline

- Model optimization: quantization, sparsity, pruning, distillation, NAS
- LLM inference bottlenecks: prefill vs decode, KV cache, batching
- PyTorch 2.0 stack: TorchDynamo, torch.export, torch.compile, FX/IR
- CUDA/TRT-LLM/Triton kernel-level performance optimization
- Multi-GPU inference: tensor, pipeline, sequence parallelism
- Profiling and debugging: Nsight Systems, Nsight Compute, PyTorch profiler
- Deployment platform thinking: automation, validation, rollback, observability
- High-probability interview Q&A and final review cheat sheet

1 Model Optimization Techniques

1.1 Quantization (Highest Probability)

Core insight: Quantization usually helps latency by reducing memory traffic first; raw compute savings are secondary unless kernels are optimized end-to-end.

PTQ vs QAT

Method	Advantages	Tradeoffs
PTQ (Post-Training Quantization)	Fast to apply, low engineering overhead	Higher accuracy risk, calibration quality matters
QAT (Quantization-Aware Training)	Better accuracy retention at low bit widths	Requires retraining and more MLOps effort

Why quantization may fail to improve latency:

- Unsupported INT8/FP8 kernels force fallback to FP16/FP32
- Dequantize/requantize boundaries add memory and launch overhead
- Fusion opportunities are lost due to mixed dtypes
- Workload is already compute-saturated at target batch size
- Poor calibration causes unstable activations and recovery logic

1.2 Sparsity and Pruning

Type	Description	Practical impact
Unstructured sparsity	Removes individual weights	Often limited speedup unless specialized sparse kernels exist
Structured sparsity	Removes heads/channels/blocks	Better hardware mapping and real latency wins

Interview signal: “Sparsity is only useful when compiler + kernel + hardware path is sparse-aware.”

1.3 Distillation and NAS

- Distillation compresses quality into smaller/faster student models
- NAS can discover hardware-friendly blocks but has high search cost
- For production, emphasize **search cost vs deployment ROI**

2 LLM Inference Optimization

2.1 Prefill vs Decode

Phase	Primary bottleneck	Best optimization levers
Prefill	GEMM compute + activation memory	Better batching, tensor core utilization, graph capture, fusion
Decode	KV-cache band-width/latency	Paged KV cache, attention kernel fusion, reduced precision cache, continuous batching

2.2 Parallelism and Sharding

- Tensor parallelism: split matrix dimensions across GPUs
- Pipeline parallelism: split layers across stages
- Sequence parallelism: split sequence/tokens across devices
- Data parallel inference: replicate model and route requests

Scaling blockers:

- NCCL communication overhead dominates after a point
- Interconnect mismatch (PCIe vs NVLink vs InfiniBand)
- Micro-batch too small to hide communication
- Host-side scheduling and data pipeline stalls

3 GPU Architecture and Profiling

3.1 Performance Limits

Constraint	Typical cause in LLM inference
Memory bandwidth	Large weights + KV-cache streaming each decode step
Compute throughput	Dense GEMMs and attention math in prefill
Kernel launch overhead	Too many tiny kernels or poor fusion
Low occupancy	Register/shared-memory pressure limits active warps

3.2 Essential Tools

Tool	What you use it for
Nsight Systems	End-to-end timeline, launch gaps, communication overlap
Nsight Compute	Kernel-level metrics: occupancy, memory transactions, tensor core usage
PyTorch Profiler	Operator hotspots, shape-dependent regressions, eager vs compiled comparison
nvidia-smi/DCGM	Device health, utilization, memory pressure, throttling signals

Benchmark hygiene checklist:

- Warmup iterations before measurement
- Synchronize correctly for GPU timing
- Report p50/p95 and throughput together
- Fix prompt length, output length, and batch policy
- Separate prefill and decode metrics

4 PyTorch 2.0 to Deployment Stack

- TorchDynamo captures graphs from eager Python programs
- `torch.export` creates a normalized, portable graph representation
- `torch.compile` applies backend optimizations and fusion
- Exported graphs feed TensorRT/TRT-LLM style automated deployment pipelines

Great interview framing: “I use graph capture to reduce Python overhead, unlock fusion, and produce stable artifacts for automated deployment and regression testing.”

5 High-Probability Interview Q&A (Role Specific)

Question	Strong answer (concise)
How would you optimize LLM inference end-to-end?	Baseline first, split prefill/decode metrics, find top kernels with Nsight, optimize memory path (KV cache + fusion), then validate quality and rollback safety.
Why does GPU utilization look low during serving?	Decode is often memory-bound and request-driven; low SM utilization can still be optimal if latency is constrained. Check memory BW, kernel launch gaps, and batching policy.
When does quantization hurt?	If kernels fall back, if dtype boundaries add conversion overhead, or if quality drops force rework. I verify kernel coverage and quality gates before rollout.
TorchDynamo vs torch.export?	Dynamo captures runtime graphs from Python execution; export gives a cleaner, standardized graph for stable deployment transformations.
How do you debug scaling stalls after adding GPUs?	Measure compute/comm overlap, inspect NCCL traces, verify topology and message sizes, and adjust parallelism mode and micro-batch size.
What makes attention kernels fast?	Coalesced memory access, fused ops, low overhead KV-cache reads/writes, and good tensor core utilization for compatible shapes/dtypes.
How do you choose tensor vs pipeline parallelism?	Tensor parallel for large GEMMs and balanced layers; pipeline for very deep models or memory limits. Choice depends on interconnect and latency target.
How do you ensure safe deployment of optimizations?	Keep golden baselines, automated accuracy/perf gates, versioned artifacts, canary rollout, and fast rollback if SLOs or quality drift regress.
What would you optimize in TRT/TRT-LLM integration?	Graph lowering coverage, kernel selection heuristics, KV-cache policies, and reproducible benchmark harnesses across model families.
How do you reason about roofline for transformers?	Decode tends toward memory roof due to KV traffic; pre-fill can approach compute roof with larger batches and fused GEMM-heavy paths.
How do you improve user-perceived latency?	Continuous batching, token streaming, smart scheduling, and prioritizing p95 TTFT/token latency over only peak throughput.
What tests are mandatory for optimization code?	Numerical parity tolerances, shape/dtype coverage, regression benchmarks, and stress tests for long-context KV-cache behavior.

6 System Design Prompts You Should Practice

1. Design an automated model optimization service from PyTorch model input to TensorRT/TRT-LLM deployment artifact.
2. Design multi-tenant LLM serving with dynamic batching, per-request QoS, and safe rollout.
3. Design a profiling pipeline that automatically flags kernel regressions after code changes.
4. Design model sharding strategy for long-context inference across 8+ GPUs.

7 Software Engineering Signals for This Role

- Strong debugging and performance analysis loop
- Modular interfaces across model, compiler, kernel, runtime layers
- Test design that catches numerical and performance regressions
- Clear documentation and reproducible benchmark methodology

8 Final Table Cheat Sheet: What to Review

Topic	Priority	What to be ready to explain	Detailed explanation to prepare
Quantization (INT8/FP8, PTQ/QAT)	High	Accuracy tradeoffs, kernel coverage, why latency may not improve	Explain that speedup comes only when the full path is quantized: graph lowering, kernels, and memory format. Be ready to discuss calibration datasets, sensitive ops (LayerNorm/Softmax), and fallback overhead from dequantize/requantize boundaries.
KV cache and attention optimization	High	Paged KV, decode bottlenecks, memory layout and bandwidth implications	Decode is usually memory-bound because each new token reads prior K/V states. Cover paged KV cache, contiguous block layout, cache precision choices, and fused attention kernels to reduce memory traffic and kernel launch overhead.

Topic	Priority	What to be ready to explain	Detailed explanation to prepare
PyTorch 2.0 compiler stack	High	TorchDynamo, <code>torch.export</code> , <code>torch.compile</code> , graph capture value	Clarify differences: TorchDynamo captures runtime graphs, <code>torch.export</code> creates stable deployable graphs, and <code>torch.compile</code> applies back-end optimizations/fusion. Explain why this reduces Python overhead and improves portability to deployment backends.
TensorRT/TRT-LLM basics	High	Deployment path, engine building, optimization and benchmarking workflow	Walk through model export, graph transformation, engine build/tactic selection, runtime execution, and regression validation. Mention dynamic shapes, precision configuration, plugin coverage, and performance-quality checks before rollout.
CUDA/Triton kernel fundamentals	High	Occupancy, memory coalescing, fusion, tensor core-friendly tiling	Be ready to reason from bottleneck to kernel design: choose tile sizes, manage shared memory/register pressure, maximize coalesced loads, and map compute to tensor cores. Explain trade-offs between occupancy and per-thread resource usage.
Distributed inference parallelism	High	Tensor/pipeline/sequence parallelism and communication bottlenecks	Explain when to choose each parallelism mode based on model size, latency target, and topology. Discuss all-reduce/all-gather costs, overlap strategies, micro-batch tuning, and why scaling can plateau when communication dominates compute.

Topic	Priority	What to be ready to explain	Detailed explanation to prepare
Profiling toolchain	High	Nsight Systems vs Nsight Compute vs PyTorch profiler usage	Present a profiling workflow: use Nsight Systems for end-to-end timeline and stalls, Nsight Compute for kernel metrics (occupancy, memory transactions, tensor core utilization), and PyTorch profiler for operator-level hotspots tied to model code.
GPU architecture essentials	Medium	SMs, warps, memory hierarchy, compute vs bandwidth limits	Explain how warp scheduling hides latency, how memory hierarchy affects throughput, and why roofline thinking helps classify kernels as compute- or memory-bound. Connect this directly to transformer prefill (more compute-heavy) vs decode (more bandwidth-bound).
Benchmark methodology	High	Warmup, synchronization, p50/p95, TTFT, tokens/sec, fair comparisons	Be precise on measurement hygiene: fixed prompt/output lengths, warmup passes, synchronized timing, percentile reporting, and separate prefill/decode analysis. Explain that fair comparisons require equal batching policy, model config, and precision settings.
Reliability and deployment safety	Medium	Canary rollout, rollback, artifact versioning, validation gates	Describe production safeguards: versioned optimization artifacts, automated numerical/performance gates, canary release with SLO monitoring, and deterministic rollback. Include how to track drift and regressions across model and runtime updates.

Final takeaway: For this NVIDIA role, your strongest interview narrative is: *profile first, optimize where bottlenecks are real, and ship safe automated deployment at scale.*